СПбГУ · МКН · СП · 1 КУРС

Математические основы алгоритмов

Конспект лекций

Лектор: Косолобов Д. А.

Составители: ....

Санкт-Петербург, 2026

Содержание

1 Быстрое преобразование Фурье (FFT). 2

2 TODO 5

3 Префиксные суммы (префиксное накопление, операция `scan`). Сумматор Ладнера–Фишера. 5

4 TODO 10

5 Ранжирование списка во внешней памяти. Линейное программирование 10

6 TODO 24

7 TODO 24

8 TODO 24

9 TODO 24

10 TODO 24

11 TODO 24

12 TODO 24

13 TODO 24

14 TODO 24

15 TODO 24

Вторая часть курса

1. Быстрое преобразование Фурье (FFT).

Свёртка и умножение многочленов

Определение 1

Пусть заданы последовательности чисел $a_0, \dots, a_{n-1}$ и $b_0, \dots, b_{n-1}$. Определим последовательность $c_0, \dots, c_{2n-2}$ по формуле

$$c_k = \sum_{t=0}^{n-1} a_t\, b_{k-t}, \qquad k = 0, \dots, 2n-2,$$

(считаем $a_i = b_i = 0$ при $i \notin [0, n-1]$). Эта операция называется **инволюцией (свертткой)**:

$$c = a * b.$$

Умножение многочленов

Рассмотрим многочлены:

$$A(x) = a_0 + a_1 x + \dots + a_{n-1} x^{n-1},$$

$$B(x) = b_0 + b_1 x + \dots + b_{n-1} x^{n-1}.$$

Тогда их произведение:

$$C(x) = A(x)B(x) = c_0 + c_1 x + \dots + c_{2n-2} x^{2n-2},$$

где коэффициенты c_k задаются инволюцией.

Сложность

- Наивное умножение: $O(n^2)$.
- Карацуба: $O\big(n^{\log_2 3}\big)$.
- FFT (быстрое преобразование Фурье): $O(n \log n)$.

Идея FFT-умножения

Хотим быстро вычислить свёртку (а значит и произведение многочленов). Для этого:

1. Выбираем число точек N так, чтобы $N \geq 2n - 1$ и обычно $N = 2^t$. Коэффициенты A, B дополняем нулями до длины N.
2. Быстро считаем значения в N точках:

   $$A(\omega_k), \quad B(\omega_k), \qquad k = 0, \dots, N-1,$$

   где ω — примитивный N-й корень из единицы.
3. Перемножаем покомпонентно:

   $$y_k = A(\omega_k)\, B(\omega_k).$$

4. По значениям y_k восстанавливаем коэффициенты C обратным FFT.

Ключевой факт: можно выбрать точки ω_k так, что и «вычисление значений», и «восстановление коэффициентов» делаются за $O(N \log N)$.

Реализация FFT

Далее считаем, что $N = 2^t$. Если исходная длина не степень двойки, дополняем нулями.

Прямое FFT: из коэффициентов в значения

Выбираем

$$\omega = e^{2\pi i/N} = \cos\frac{2\pi}{N} + i\sin\frac{2\pi}{N}, \qquad \omega_k = \omega^k.$$

Нужно вычислить

$$y_k = A(\omega^k) = \sum_{j=0}^{N-1} a_j(\omega^k)^j, \qquad k = 0, \dots, N-1.$$

Разобьём A на чётные и нечётные коэффициенты:

$$A_0(x) = a_0 + a_2 x + \dots + a_{N-2}x^{\frac{N}{2}-1}, \qquad A_1(x) = a_1 + a_3 x + \dots + a_{N-1}x^{\frac{N}{2}-1}.$$

Тогда

$$A(x) = A_0(x^2) + xA_1(x^2).$$

Подставим $x = \omega^k$:

$$A(\omega^k) = A_0(\omega^{2k}) + \omega^k A_1(\omega^{2k}).$$

Обозначим

$$y_k^{(0)} = A_0(\omega^{2k}), \qquad y_k^{(1)} = A_1(\omega^{2k}).$$

Так как значения ω^{2k} пробегают только $N/2$ различных точек, достаточно посчитать $y_k^{(0)}, y_k^{(1)}$ для $k = 0, \dots, \frac{N}{2} - 1$ (рекурсивно), а затем «склеить» ответы:

$$y_k = y_k^{(0)} + \omega^k\, y_k^{(1)}, \qquad y_{k+\frac{N}{2}} = y_k^{(0)} - \omega^k\, y_k^{(1)}.$$

(Здесь использовано $\omega^{k+N/2} = -\omega^k$.)

Оценка времени прямого FFT

На каждом уровне рекурсии решаем 2 подзадачи размера $N/2$ и делаем склейку за $O(N)$:

$$T(N) = 2T\left(\frac{N}{2}\right) + O(N) = O(N \log N).$$

Обратное FFT: из значений в коэффициенты

Прямое преобразование можно записать матрично:

$$\begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega^1 & (\omega^1)^2 & \cdots & (\omega^1)^{N-1} \\ 1 & \omega^2 & (\omega^2)^2 & \cdots & (\omega^2)^{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & (\omega^{N-1})^2 & \cdots & (\omega^{N-1})^{N-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{N-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{pmatrix}.$$

Это матрица Вандермонда F_ω.

Утверждение 1

Выполнено
$$F_{\omega^{-1}} F_\omega = N \cdot \mathbf{I}.$$

Доказательство. Элемент в позиции (i, j) равен
$$\sum_{k=0}^{N-1} (\omega^i)^k (\omega^{-j})^k = \sum_{k=0}^{N-1} (\omega^{i-j})^k.$$

Если $i \neq j$, то $q := \omega^{i-j} \neq 1$, и по формуле суммы геометрической прогрессии
$$\sum_{k=0}^{N-1} q^k = \frac{1 - q^N}{1 - q}$$
получаем
$$\sum_{k=0}^{N-1} (\omega^{i-j})^k = \frac{1 - (\omega^{i-j})^N}{1 - \omega^{i-j}} = \frac{1 - (\omega^N)^{i-j}}{1 - \omega^{i-j}} = \frac{1 - 1}{1 - \omega^{i-j}} = 0.$$
Если $i = j$, то
$$\sum_{k=0}^{N-1} (\omega^{i-j})^k = \sum_{k=0}^{N-1} 1 = N.$$

■

Следовательно, обратное преобразование — это прямое FFT с корнем ω^{-1}, после чего нужно поделить все коэффициенты на N:
$$a = \frac{1}{N} F_{\omega^{-1}} y.$$

Сложность умножения многочленов через FFT

Нужно:

- одно прямое FFT для A,
- одно прямое FFT для B,
- покомпонентное умножение ($O(N)$),
- одно обратное FFT.

Итого $O(N \log N)$, где N — ближайшая степень двойки, не меньшая $2n - 1$.

При этом $2n - 1 \leq N < 2(2n - 1) < 4n$, то есть $N = \Theta(n)$, поэтому итоговая сложность равна $O(n \log n)$.

Обобщение: NTT (Number-Theoretic Transform)

Вместо $\mathbb{C}$ можно работать в кольце/поле R, если:

- в R существует элемент ω порядка N (примитивный N-й корень из единицы);
- число N обратимо в R (нужно для деления на N в обратном преобразовании).

Пример (поле $\mathbb{Z}_p$)

Пусть $R = \mathbb{Z}_p$, где p — простое, и

$$N = 2^k, \qquad p = c \cdot 2^k + 1.$$

Тогда $p - 1$ кратно N, и существует первообразный корень g по модулю p (порождает мультипликативную группу порядка $p - 1$). Можно взять

$$\omega \equiv g^c \pmod p,$$

тогда ω имеет порядок N, и NTT работает полностью по модулю p.

Приложение: смысл коэффициентов DFT

Пусть дан сигнал (последовательность)

$$a_0, a_1, \ldots, a_{N-1}.$$

Определим дискретное преобразование Фурье:

$$y_k = \sum_{t=0}^{N-1} a_t \, \omega^{kt}, \qquad k = 0, \ldots, N-1, \quad \text{где} \;\; \omega = e^{2\pi i/N}.$$

Так как

$$\omega^{kt} = \cos\left(\frac{2\pi k}{N} t\right) + i \sin\left(\frac{2\pi k}{N} t\right),$$

то

$$\Re(y_k) = \sum_{t=0}^{N-1} a_t \cos\left(\frac{2\pi k}{N} t\right), \qquad \Im(y_k) = \sum_{t=0}^{N-1} a_t \sin\left(\frac{2\pi k}{N} t\right).$$

Эти суммы — скалярные произведения сигнала с косинусом и синусом частоты k. Если в сигнале сильно выражена гармоника частоты k, то вклады «складываются», и модуль $|y_k|$ получается большим; если такой частоты нет — вклады в основном взаимно компенсируются и $|y_k|$ близок к нулю.

2. TODO

3. Префиксные суммы (префиксное накопление, операция `scan`). Сумматор Ладнера–Фишера.

Задача (prefix sums). Построить вычислительную схему, которая принимает на вход n чисел $x_0, \ldots, x_{n-1}$ и (используя только бинарную операцию “+”) выдаёт числа $y_0, \ldots, y_{n-1}$, где

$$y_i = \sum_{j=0}^{i} x_j, \qquad i = 0, \ldots, n-1.$$

Схема Ладнера–Фишера

Пусть n чётное.

Сначала попарно складываем входы:

$$s_i = x_{2i} + x_{2i+1}, \qquad i = 0, \ldots, \frac{n}{2} - 1.$$

Рекурсивно считаем префиксные суммы для s:

$$p_i = \sum_{j=0}^{i} s_j.$$

Тогда выходы восстанавливаются так:

$$y_{2i+1} = p_i, \qquad y_{2i} = \begin{cases} x_0, & i = 0, \\ p_{i-1} + x_{2i}, & i \geq 1. \end{cases}$$

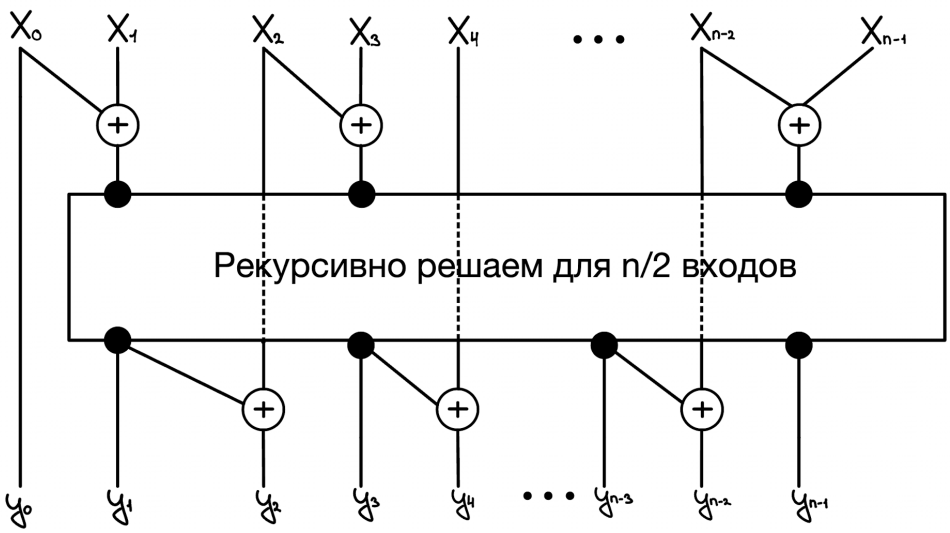

Рис. 1: Схема Ладнера–Фишера для префиксных сумм

Оценки (глубина и размер).

$T(n) = 2 + T\left(\frac{n}{2}\right)$, $T(1) = 0 \Rightarrow T(n) = 2\log n$ $(n = 2^m)$, $\quad S(n) = (n-1) + S\left(\frac{n}{2}\right) = 2n - \log n - 2.$

Идея хорошо ложится на GPU (много независимых операций на каждом уровне).

Замечание. Операция `scan` (префиксное накопление) определена для любой **ассоциативной** бинарной операции $\odot$ на множестве S (то есть для **полугруппы** $(S, \odot)$):

$$y_i = x_0 \odot x_1 \odot \cdots \odot x_i.$$

Примеры операций $\odot$:

$$(\mathbb{R}, +), \quad (\mathbb{R}, \min), \quad (\mathbb{R}, \max), \quad (\mathbb{R}^{k \times k}, \cdot) \text{ (умножение матриц).}$$

Псевдокод `scan`

Algorithm 1 `scan` (in-place), inclusive, по доске

Require: массив $x[0..n-1]$, где $n = 2^m$

```
                                  ▷ Upsweep: накапливаем суммы на концах блоков

for d = 0 to log_2 n − 1 do ▷ для фиксированного d все операции внутреннего цикла по k выполняются параллельно
    for k = 0 to n − 1 step 2^{d+1} do
        x[k + 2^{d+1} − 1] = x[k + 2^{d+1} − 1] + x[k + 2^d − 1]
                  ▷ Downsweep: добавляем префикс предыдущего блока к левому подблоку
for d = log_2 n − 1 downto 0 do      ▷ для фиксированного d все операции внутреннего цикла по k выполняются параллельно
    for k = 0 to n − 1 step 2^{d+1} do
        if k > 0 then
            x[k + 2^d − 1] = x[k + 2^d − 1] + x[k − 1]
```

Пример 1: Сведения к `scan`: рекурсия вида $c_{k+1} = a_k + b_k c_k$

Даны числа $a_0, \dots, a_{n-1}$ и $b_0, \dots, b_{n-1}$. Нужно найти $c_0, \dots, c_n$ такие, что

$$c_0 = 0, \qquad c_{k+1} = a_k + b_k\, c_k.$$

Определим строковый вектор

$$C_k = (1,\ c_k).$$

Тогда

$$C_{k+1} = C_k\, X_k, \qquad X_k = \begin{pmatrix} 1 & a_k \\ 0 & b_k \end{pmatrix}.$$

Отсюда

$$C_1 = C_0 X_0, \quad C_2 = C_0 X_0 X_1, \quad C_3 = C_0 X_0 X_1 X_2, \ \dots$$

То есть достаточно посчитать префиксные произведения матриц $X_0, X_1, \dots, X_{n-1}$ (умножение матриц ассоциативно), и это делается через `scan` за $O(\log n)$ времени в параллельной модели.

Замечание. В примере с матрицами внутри умножения матриц одновременно используются две операции над элементами:

$$(A \cdot B)_{ij} = \bigoplus_k (A_{ik} \odot B_{kj}).$$

Поэтому обобщение работает не для одной операции, а для **полукольца** $(S, \oplus, \odot)$: требуется, чтобы

- $\oplus$ ассоциативна (обычно коммутативна) и имеет нейтральный элемент 0;
- $\odot$ ассоциативна и имеет нейтральный элемент 1;
- $\odot$ дистрибутивна относительно $\oplus$: $a \odot (b \oplus c) = (a \odot b) \oplus (a \odot c)$ и $(a \oplus b) \odot c = (a \odot c) \oplus (b \odot c)$;
- 0 поглощает умножение: $0 \odot a = a \odot 0 = 0$.

Важно: **порядок операций существенен** — в роли "сложения" стоит $\oplus$ (левая операция), а в роли "умножения" стоит $\odot$ (правая операция).

Примеры полукольца $(S, \oplus, \odot)$:

$$(\{0,1\},\ \vee,\ \wedge), \qquad (\{0,1\},\ \oplus,\ \wedge), \qquad (\mathbb{R} \cup \{+\infty\},\ \min,\ +).$$

Сумматор Ладнера–Фишера и переносы

Определение 2

Сумматор — это вычислительная схема над булевыми операциями $(\vee, \wedge, \oplus, \neg, \dots)$, которая принимает на вход биты

$$x_0, \dots, x_{n-1}, \qquad y_0, \dots, y_{n-1}$$

и выдаёт биты

$$z_0, \dots, z_n$$

такие, что $z = x + y$, где

$$x = \sum_{k=0}^{n-1} x_k 2^k, \qquad y = \sum_{k=0}^{n-1} y_k 2^k, \qquad z = \sum_{k=0}^{n} z_k 2^k.$$

Биты переноса и сведение к `scan`

Пусть c_k — перенос *в* разряд k (по определению $c_0 = 0$). Тогда для $k = 0, \dots, n-1$ выполняется формула переноса:

$$c_{k+1} = (x_k \wedge y_k)\ \vee\ (x_k \wedge c_k)\ \vee\ (y_k \wedge c_k).$$

Её удобно переписать в виде

$$c_{k+1} = a_k\ \vee\ (b_k \wedge c_k), \qquad a_k := x_k \wedge y_k, \quad b_k := x_k \vee y_k.$$

Если все переносы $c_0, \dots, c_n$ известны, то ответ восстанавливается по формулам

$$z_k = x_k \oplus y_k \oplus c_k \quad (k = 0, \dots, n-1), \qquad z_n = c_n.$$

Ключевая идея: рекуррента

$$c_{k+1} = a_k\ \vee\ (b_k \wedge c_k)$$

имеет вид “аффинного” преобразования, которое можно упаковать в матрицу. Положим

$$C_k = (1,\ c_k), \qquad X_k = \begin{pmatrix} 1 & a_k \\ 0 & b_k \end{pmatrix}.$$

Тогда

$$C_{k+1} = C_k X_k,$$

где операции в матрице понимаются над булевым полукольцом $(\{0,1\},\ \vee,\ \wedge)$. Следовательно, нужно посчитать префиксные произведения

$$C_k = C_0 X_0 X_1 \cdots X_{k-1},$$

что делается через `scan` за $O(\log n)$ времени (в параллельной модели).

Модель PRAM

Определение 3

PRAM (Parallel Random Access Machine) — идеализированная модель параллельных вычислений:

- есть **общая глобальная память**;
- есть (теоретически) **неограниченное число процессоров**, каждый процессор — RAM с собственным номером $p = 1, 2, \ldots$;
- процессоры работают **синхронно**: за единицу времени выполняют одну инструкцию.

Разрешение конфликтов доступа к памяти

Для чтения/записи различают режимы **concurrent/exclusive**:

- **concurrent read** — несколько процессоров могут одновременно читать одну ячейку;
- **concurrent write** — несколько процессоров могут одновременно писать в одну ячейку; в модели CRCW нужно отдельно задавать правило разрешения конфликта (например, *arbitrary*: остаётся одно из записанных значений);
- **exclusive** — если несколько процессоров обращаются к одной ячейке "одновременно", это запрещено (поведение не определено/"мусор").

Стандартные варианты PRAM:

$$\text{CRCW PRAM}, \quad \text{CREW PRAM}, \quad \text{ERCW PRAM}, \quad \text{EREW PRAM}.$$

Класс NC

NC — класс задач, которые решаются на PRAM за время $(\log n)^{O(1)}$ при использовании *полиномиального* числа процессоров $n^{O(1)}$, где n – размер входа.

Как в PRAM посчитать `scan` на p процессорах

Пусть дан массив $A[0..n-1]$ и p процессоров. Разобьём массив на блоки длины $m = n/p$ (для простоты считаем $p \mid n$).

1. Каждый процессор i считает **локальный scan** на своём блоке $A[im..(i+1)m-1]$ последовательно за время $O(m) = O(n/p)$. Обозначим сумму блока как T_i (последний элемент локального scan).
2. Считаем **scan по массиву блок-сумм** $T_0, T_1, \ldots, T_{p-1}$ за время $O(\log p)$. Пусть
$$P_i = \sum_{j=0}^{i-1} T_j$$
— сумма всех предыдущих блоков (смещение для блока i).
3. Каждый процессор прибавляет смещение P_i ко всем элементам своего блока (снова $O(n/p)$ времени).

Итого время:

$$O\left(\frac{n}{p} + \log p\right).$$

В частности, при $p = \Theta\left(\frac{n}{\log n}\right)$ получаем $O(\log n)$ времени.

Пример 2: Сортировка подсчётом (Counting sort) в PRAM

Дан массив $a[0..n-1]$, причём $a[k] \in \{0, \dots, U\}$, где $U = O(1)$. Есть p процессоров, нумеруем их $i = 0, 1, \dots, p-1$. Каждый процессор обрабатывает блок длины $m = \frac{n}{p}$.

- Процессор i строит таблицу частот $c_i[0..U]$, где

$$c_i(x) = \#\{\, k \in [im, (i+1)m - 1] \mid a[k] = x \,\}, \qquad m = \frac{n}{p}.$$

  Это занимает $O(\frac{n}{p})$ времени.
- Далее соберём все частоты в один массив длины $(U+1)p$ в порядке

$$C[xp + i] = c_i(x), \qquad x = 0, \dots, U, \quad i = 0, \dots, p-1,$$

  то есть

$$C = \big(c_0(0), c_1(0), \dots, c_{p-1}(0),\ c_0(1), \dots, c_{p-1}(1),\ \dots,\ c_{p-1}(U)\big).$$

  Посчитаем `scan` (префиксные суммы) по C и получим массив P.
- После этого процессор i знает, сколько элементов значения x находится у процессоров с номерами $\le i$ (это $P[xp+i]$), а значит знает и **диапазон позиций** в выходном массиве для своих элементов значения x:

$$\text{элементы } x \text{ из блока } i \text{ кладём в } \big[\, P[xp+i] - c_i(x),\ P[xp+i) \,\big)$$

  И просто выписываем элементы в эти позиции.

Время. Локальный подсчёт частот занимает $O(n/p)$. `scan` по массиву длины $(U+1)p$ занимает $O(\log((U+1)p)) = O(\log p)$, так как $U = O(1)$. Итого время:

$$O\left(\frac{n}{p} + \log p\right).$$

4. TODO

5. Ранжирование списка во внешней памяти. Линейное программирование

5.1. Задача

Дан список, хранящийся в массиве $A[0..N-1]$ в произвольном порядке. У каждого узла x есть поля

$$x.\texttt{next}, \qquad x.w.$$

Также в реализации удобно хранить поле $x.\texttt{id}$, причём изначально

$$A[i].\texttt{id} = i.$$

Замечание 1

Ниже в рассуждениях мы иногда будем писать $\mathrm{pred}(x)$ для предшественника вершины x в текущем списке. Это *не поле структуры данных*, а только удобное математическое обозначение для анализа и для псевдокода.

Исходная задача состоит в том, чтобы для каждой вершины x определить её номер в списке, считая от головы:

$$1, 2, \ldots, N.$$

Эквивалентно, нужно упорядочить вершины массива A в порядке их следования вдоль списка.

Для решения этой задачи удобно рассмотреть более общую подзадачу: для каждой вершины x вычислить сумму весов на префиксе списка до неё, то есть

$$\mathrm{pref}(x) = \sum_{y \text{ от головы списка до } x} y.w.$$

После работы алгоритма в поле $x.w$ будет записано значение $\mathrm{pref}(x)$.

Если изначально у всех вершин $w = 1$, то $\mathrm{pref}(x)$ есть просто номер вершины x в списке, считая от головы:

$$1, 2, \ldots, N.$$

5.2. Идея сжатия списка

Пусть S — множество удаляемых узлов, такое что никакие два соседних по списку узла не удаляются одновременно:

$$\forall x \in S \quad \mathrm{pred}(x) \notin S, \qquad |S| = \Theta(N).$$

Тогда все узлы из S можно удалить *независимо друг от друга*: для каждого $x \in S$ выполняем

$$\mathrm{pred}(x).\texttt{next} = x.\texttt{next}, \qquad \mathrm{pred}(x).w \mathrel{+}= x.w.$$

После этого каждый оставшийся узел хранит суммарный вес некоторого подряд идущего блока исходных вершин.

На получившемся более коротком списке мы рекурсивно решаем *ту же самую задачу*: вычисляем префиксные суммы этих объединенных весов.

После возврата из рекурсии удалённые вершины нужно восстановить и дописать для них ответы

5.3. Как выбрать множество S

Каждой вершине независимо присваиваем случайный бит

$$x.r \in \{0, 1\}, \qquad \Pr[x.r = 0] = \Pr[x.r = 1] = \frac{1}{2}.$$

После этого выбираем

$$x \in S \iff x.r = 1 \text{ и } \mathrm{pred}(x).r = 0.$$

То есть в S попадают вершины, которые являются “правым концом” шаблона 01 вдоль списка.

Почему это корректно

Если $x \in S$, то по определению $\text{pred}(x).r = 0, \qquad x.r = 1$.

Значит, $\text{pred}(x)$ не может одновременно лежать в S, потому что для этого у него должен был бы быть бит 1.

Следовательно, $\forall x \in S\colon\ \text{pred}(x) \notin S$.

Размер множества S

Для внутренней вершины вероятность попасть в S равна

$$\Pr[x \in S] = \Pr[\text{pred}(x).r = 0 \wedge x.r = 1] = \frac{1}{4}.$$

Поэтому $\mathbb{E}|S| \approx \frac{N}{4}$,

а после одного шага остаётся примерно $N - |S| \approx \frac{3N}{4}$ вершин.

5.4. Рекурсивная схема алгоритма

Следующий псевдокод — это *не* реализация во внешней памяти, а только удобная запись логики алгоритма, где разрешено использовать обозначение $\text{pred}(x)$.

Algorithm 2 PrefixSums-Conceptual(A)

Require: Текущий список A
Ensure: Для каждой вершины x значение $x.w$ равно сумме весов на префиксе до x

```
if |A| ≤ 1 then
    return A
Для каждой вершины x ∈ A независимо выбрать случайный бит x.r ∈ {0, 1}
S = { x ∈ A | pred(x) существует, pred(x).r = 0, x.r = 1 }
                                        ▷ Сжимаем список, удаляя вершины из S
for all x ∈ S do
    p = pred(x)
    p.w = p.w + x.w
    p.next = x.next
A′ = A \ S
PrefixSums-Conceptual(A′)
                                        ▷ Восстанавливаем удалённые вершины
for all x ∈ S в порядке, обратном удалению do
    p = pred(x)
    p.next = x
    p.w = p.w − x.w                     ▷ теперь p.w = префиксная сумма до p
    x.w = x.w + p.w                     ▷ теперь x.w = префиксная сумма до x
return A
```

5.5. Реализация через сортировки во внешней памяти

Главная цель — заменить случайные обращения к указателям на несколько сортировок и линейных проходов.

Замечание 2

Ниже для краткости не выписана отдельная обработка головы списка, хвоста списка и значения NULL в поле `next`. В реальной реализации эти пограничные случаи нужно обрабатывать отдельно (например, специальными фиктивными вершинами или отдельными ветками в коде).

Храним идентификаторы

У каждого узла есть поле

$$x.\texttt{id}, \qquad A[i].\texttt{id} = i.$$

Это позволяет после перестановок и сортировок восстановить "исходную" вершину.

Algorithm 3 EM_PrefixSums(A)

Require: Массив вершин A, отсортированный по `id`
Ensure: Для каждой вершины x значение $x.w$ равно сумме весов на префиксе до x

```
if |A| ≤ 1 then
    return A
(Â, S) = Compress(A)                ▷ Â — сжатый список, S — удалённые вершины
Â = EM_PrefixSums(Â)                ▷ рекурсивно решаем ту же задачу на объединенных весах
A = Restore(Â, S, |A|)
return A
```

Сортируем по `next`

Пусть массив A отсортирован по `id`. Тогда строим

$$\widetilde{A} = \mathrm{sort}(\{x \in A\} \text{ по } x.\texttt{next}).$$

После такой сортировки запись $\widetilde{A}[i]$ стоит напротив той вершины, в которую она указывает. Поэтому при одновременном просмотре массивов A и $\widetilde{A}$ можно сопоставить вершину и её предшественника.

Находим удаляемые вершины и сразу обновляем предшественников

При линейном проходе по A и $\widetilde{A}$:

$$\text{если } \widetilde{A}[i].r = 0 \text{ и } A[i].r = 1, \text{ то удаляем } A[i].$$

При этом выполняются обновления:

$$\widetilde{A}[i].w \mathrel{+}= A[i].w, \qquad \widetilde{A}[i].\texttt{next} = A[i].\texttt{next},$$

а сама удалённая вершина сохраняется в массив S.

Убираем "дубликаты" после перенаправления указателей

После удаления вершины её предшественник начинает указывать в того же преемника, в которого раньше указывала удалённая вершина. Поэтому после повторной сортировки по `next`

$$\widetilde{A} = \mathrm{sort}(\widetilde{A} \text{ по } \texttt{next})$$

появляются соседние элементы с одинаковым полем `next`.

Из каждой такой пары надо оставить только живую вершину, а удалённую выбросить. Это делается линейным проходом по отсортированному массиву: если у двух соседних элементов одинаковый `next`, то удаляется тот, у которого $r = 1$.

После этого выжившие вершины упаковываются в плотный массив длины $N - |S|$.

Algorithm 4 Compress(A)

Require: Массив $A[0..N-1]$, отсортированный по `id`
Ensure: Сжатый массив $\widehat{A}$ и массив удалённых вершин S

Для каждой вершины $x \in A$ независимо выбрать случайный бит $x.r \in \{0, 1\}$
$\widetilde{A} = \text{sort}(A \text{ по } x.\texttt{next})$
$S = [\,]$
▷ Находим вершины из шаблона 01 и сразу переносим их вес в предшественника
for $i = 0$ **to** $N - 1$ **do**
    if $\widetilde{A}[i].r = 0$ **and** $A[i].r = 1$ **then**
        $\widetilde{A}[i].w = \widetilde{A}[i].w + A[i].w$
        $\widetilde{A}[i].\texttt{next} = A[i].\texttt{next}$
        S.push_back($A[i]$)
$\widetilde{A} = \text{sort}(\widetilde{A} \text{ по } x.\texttt{next})$
▷ Упаковываем только живые вершины
$B = [\,]$
$i = 0$
while $i < N$ **do**
    if $i + 1 < N$ **and** $\widetilde{A}[i].\texttt{next} = \widetilde{A}[i+1].\texttt{next}$ **then**
        if $\widetilde{A}[i].r = 1$ **then**
            swap($\widetilde{A}[i], \widetilde{A}[i+1]$) ▷ после swap живая вершина стоит первой
        B.push_back($\widetilde{A}[i]$)
        $i = i + 2$
    else
        B.push_back($\widetilde{A}[i]$)
        $i = i + 1$
▷ Переопределяем `next`: старые `id` заменяются на новые позиции в плотном массиве
$\widetilde{B} = \text{sort}(B \text{ по } x.\texttt{next})$
$i = 0$; $j = 0$; $k = 0$
while $j < N - |S|$ **do**
    while $k < |S|$ **and** $i = S[k].\texttt{id}$ **do**
        $i = i + 1$
        $k = k + 1$
    $\widetilde{B}[j].\texttt{next} = j$
    $i = i + 1$
    $j = j + 1$
$\widehat{A} = \text{sort}(\widetilde{B} \text{ по } x.\texttt{id})$ ▷ `id` сохраняются исходными, веса w уже агрегированы
return $(\widehat{A}, S)$

Перенумерация и рекурсия

После упаковки нужно переопределить индексы `next` под новый массив, затем отсортировать вершины по `id` и рекурсивно решить задачу уже для массива размера $N - |S|$:

$$\widehat{A} = \text{sort}(\{x \in A[0..N - |S| - 1]\} \text{ по } x.\texttt{id}).$$

Восстановление

Предполагается, что:

- S хранится в порядке возрастания `id`;
- $\widehat{A}$ — результат рекурсивного решения на сжатом массиве длины $N - |S|$.

Сначала сортируем удалённые и оставшиеся вершины по полю `next`:

$$\widetilde{S} = \text{sort}(\{x \in S\} \text{ по } x.\texttt{next}), \qquad \widetilde{A} = \text{sort}(\{x \in \widehat{A}[0..N - |S| - 1]\} \text{ по } x.\texttt{next}).$$

После этого выполняем линейный проход:

Algorithm 5 Restore($\widehat{A}, S, N$)

```
1: S̃ = sort(S по x.next)
2: Ã = sort(Â[0..N − |S| − 1] по x.next)
        ▷ i идёт по старым id, k — по S в порядке id    ▷ j идёт по живым вершинам в порядке next, m — по S в порядке next
3: i = 0; j = 0; k = 0; m = 0
4: while j < N − |S| do
5:     while k < |S| and i = S[k].id do
6:         i = i + 1
7:         k = k + 1                                    ▷ пропускаем старые id удалённых вершин
8:     Ã[j].next = i        ▷ восстановили старый id вершины, на которую вёл указатель
9:     if m < |S| and Ã[j].next = S̃[m].next then    ▷ значит, между предшественником и его текущим преемником надо вернуть удалённую вершину
10:        Ã[j].next = S̃[m].id
11:        Ã[j].w = Ã[j].w − S̃[m].w                    ▷ теперь Ã[j].w = префиксная сумма до предшественника
12:        S̃[m].w = S̃[m].w + Ã[j].w                    ▷ теперь S̃[m].w = префиксная сумма до восстановленной вершины
13:        m = m + 1
14:    i = i + 1
15:    j = j + 1
16: A = sort(x ∈ (S̃ ∪ Ã[0..N − |S| − 1]) по id)
17: return A
```

5.6. Оценка сложности

Обозначим через $\text{sort}(N)$ стоимость сортировки N записей во внешней памяти.

Один уровень рекурсии состоит из:

- константного числа сортировок по полям `id` и `next`;
- константного числа линейных проходов по массивам.

Поэтому существует константа $c > 0$ такая, что работа одного уровня рекурсии не превосходит

$$c \operatorname{sort}(N).$$

Доказательство по индукции

Докажем по индукции, что

$$\mathbb{E}[T(n)] \le 4c \operatorname{sort}(n).$$

База индукции для малых n очевидна, если увеличить константу c при необходимости.

Пусть теперь утверждение верно для всех размеров меньше n. Тогда, раскладывая по возможным значениям $|S|$, получаем

$$\mathbb{E}[T(n)] = \sum_{k=0}^{n-1} \Pr[|S| = k] \cdot \mathbb{E}[T(n-k)] + c \operatorname{sort}(n).$$

Здесь для простоты суммируем по всем $k < n$, включая $k = 0$.

По предположению индукции:

$$\mathbb{E}[T(n)] \le \sum_{k=0}^{n-1} \Pr[|S| = k] \cdot 4c \operatorname{sort}(n-k) + c \operatorname{sort}(n).$$

Используем стандартную оценку

$$\operatorname{sort}(m) = \Theta\left(\frac{m}{B} \log_{M/B} \frac{m}{B}\right).$$

Тогда, поскольку $n - k \le n$,

$$\operatorname{sort}(n-k) \le \frac{n-k}{B} \log_{M/B} \frac{n}{B}$$

с точностью до постоянного множителя, уже поглощённого константой c. Следовательно,

$$\mathbb{E}[T(n)] \le 4c \log_{M/B} \frac{n}{B} \sum_{k=0}^{n-1} \Pr[|S| = k] \frac{n-k}{B} + c \operatorname{sort}(n).$$

Но

$$\sum_{k=0}^{n-1} \Pr[|S| = k](n-k) = \mathbb{E}[n - |S|].$$

Так как каждая внутренняя вершина удаляется с вероятностью $1/4$, в среднем после сжатия остаётся не более $\frac{3n}{4}$ вершин (с точностью до констант, связанных с крайними вершинами списка), то есть

$$\mathbb{E}[n - |S|] \le \frac{3n}{4}.$$

Подставляя это в предыдущую оценку, получаем

$$\mathbb{E}[T(n)] \le 4c \log_{M/B} \frac{n}{B} \cdot \frac{3n}{4B} + c \operatorname{sort}(n) \le 3c \operatorname{sort}(n) + c \operatorname{sort}(n) = 4c \operatorname{sort}(n).$$

Индукционный переход доказан, значит

$$\mathbb{E}[T(n)] = O(\operatorname{sort}(n)).$$

Итог

Таким образом, ожидаемая сложность алгоритма равна

$$\mathbb{E}[T(N)] = O(\mathrm{sort}(N)) = O\left(\frac{N}{B}\log_{M/B}\frac{N}{B}\right).$$

Предложение 1

Пусть дано корневое дерево на N вершинах. У каждой вершины v хранится её список детей в нужном порядке обхода:

```
struct { int id; int kids_num; int kids[kids_num]; }.
```

Требуется выписать все вершины в порядке обхода DFS.

Идея. Построим список, соответствующий эйлерову обходу дерева, и посчитаем на нём префиксные суммы весов.

Построение списка

Для каждого ребра (v, u), где u — ребёнок v, создаём две записи:

$$(v \to u) \qquad \text{и} \qquad (u \to v).$$

Также добавляем фиктивную стартовую запись s.

Поле `next` задаём так, чтобы эти записи шли в порядке эйлерова обхода:

- после дуги $(v \to u)$ идём либо в первую дугу из u к его первому ребёнку, либо, если детей нет, сразу в $(u \to v)$;
- после дуги $(u \to v)$ идём либо к следующему ребёнку вершины v, либо возвращаемся вверх.

Таким образом получаем список длины $2N - 1$.

Веса

Положим

$$s.w = 1,$$

$$(v \to u).w = 1 \quad \text{для дуги вниз,} \qquad (u \to v).w = 0 \quad \text{для дуги вверх.}$$

Теперь запускаем алгоритм вычисления префиксных сумм на этом списке.

Почему это работает

Единица добавляется ровно в момент первого входа в новую вершину. Поэтому:

- корень получает номер 1;
- для каждой дуги вниз $(v \to u)$ значение префиксной суммы на этой записи равно номеру вершины u в порядке DFS.

Значит, если собрать пары

$$(1, \mathrm{root}) \qquad \text{и} \qquad ((v \to u).w,\ u) \text{ для всех дуг вниз,}$$

а затем отсортировать их по первому компоненту, то вершины будут выписаны в порядке обхода DFS.

Сложность

Число записей равно $O(N)$. Построение эйлерова списка и финальная сортировка требуют константного числа сортировок и линейных проходов, а префиксные суммы на списке считаются за $O(\mathrm{sort}(N))$.

Следовательно, вся задача решается за

$$O(\mathrm{sort}(N)).$$

Линейное программирование

5.7. Общая постановка задачи ЛП

Линейное программирование (ЛП) — это задача оптимизации линейной функции при линейных ограничениях.

В общем виде задача может быть записана так:

$$\max_{x \in \mathbb{R}^n} c^T x \qquad \text{при} \qquad Ax \le b,$$

где

$$A \in \mathbb{R}^{m \times n}, \qquad b \in \mathbb{R}^m, \qquad c \in \mathbb{R}^n.$$

Неравенство $Ax \le b$ понимается **покомпонентно**, то есть

$$(Ax)_i \le b_i, \qquad i = 1, \dots, m.$$

То же самое в координатной записи:

$$\max\ c_1 x_1 + \dots + c_n x_n$$

при условиях

$$\begin{cases} a_{11}x_1 + \dots + a_{1n}x_n \le b_1, \\ a_{21}x_1 + \dots + a_{2n}x_n \le b_2, \\ \qquad\qquad \vdots \\ a_{m1}x_1 + \dots + a_{mn}x_n \le b_m. \end{cases}$$

Две наиболее употребительные записи:

$$\max\ c^T x \quad \text{при } Ax \le b,$$

— *стандартная форма*, и

$$\min\ c^T x \quad \text{при } Ax = b,\ x \ge 0,$$

— *каноническая форма.*

Обе формы эквивалентны общей задаче после несложных преобразований.

5.8. Эквивалентные преобразования задач ЛП

Многие задачи линейного программирования можно привести к одной удобной форме.

1) Минимизацию можно заменить максимизацией

$$\min\ c^T x \quad \Longleftrightarrow \quad \max\ (-c)^T x.$$

2) Ограничение вида $\geq$ можно заменить ограничением вида $\leq$ Если

$$a_k^T x \geq b_k,$$

то, умножив обе части на -1, получаем

$$-a_k^T x \leq -b_k.$$

3) Равенство можно заменить двумя неравенствами Если

$$a_k^T x = b_k,$$

то это эквивалентно системе

$$\begin{cases} a_k^T x \leq b_k, \\ -a_k^T x \leq -b_k. \end{cases}$$

4) Свободную переменную можно заменить разностью двух неотрицательных Если переменная x_j может быть любого знака, то вводят

$$x_j^+ \geq 0, \qquad x_j^- \geq 0,$$

и полагают

$$x_j = x_j^+ - x_j^-.$$

Это стандартный способ перейти к задаче с неотрицательными переменными.

5) Неравенство $\leq$ можно превратить в равенство Если имеется ограничение

$$a_k^T x \leq b_k,$$

то вводится **добавочная переменная** (slack variable)

$$s_k \geq 0,$$

такая что

$$a_k^T x + s_k = b_k.$$

Смысл переменной s_k: это *остаток ресурса* или *запас* до границы ограничения.

5.9. Приведение к канонической форме

Одна из самых удобных форм:

$$\min\ c^T x \qquad \text{при} \qquad Ax = b, \qquad x \geq 0.$$

Чтобы привести к ней произвольную линейную задачу, обычно делают так:

1. при необходимости заменяют максимум на минимум или наоборот;
2. ограничения вида $\geq$ заменяют на $\leq$, умножая на -1;
3. неравенства заменяют равенствами с помощью добавочных переменных;
4. каждую свободную переменную заменяют разностью двух неотрицательных.

5.10. Геометрическая интерпретация

Пусть строки матрицы A равны

$$a_1^T, \dots, a_m^T.$$

Тогда каждое ограничение

$$a_k^T x \le b_k$$

задаёт **полупространство**.

Следовательно, допустимое множество

$$P = \{x \in \mathbb{R}^n \mid Ax \le b\}$$

есть пересечение полупространств, то есть **выпуклый многогранник**.

Целевая функция

$$c^T x$$

задаёт семейство гиперплоскостей

$$c^T x = d.$$

При изменении d эта гиперплоскость сдвигается параллельно самой себе. Задача ЛП состоит в том, чтобы найти такую гиперплоскость, которая:

1. ещё пересекает допустимое множество;
2. при этом даёт наибольшее (или наименьшее) значение целевой функции.

Пример 3: План производства

Предприятие выпускает два вида продукции:

$$x_1 = \text{число столов}, \qquad x_2 = \text{число стульев}.$$

Прибыль:

$$40 \text{ у.е. за стол}, \qquad 30 \text{ у.е. за стул}.$$

Есть два ресурса:

- древесина: не более 18 единиц;
- рабочее время: не более 10 часов.

На производство одной единицы продукции требуется:

	стол (x_1)	стул (x_2)
древесина	3	2
время	2	1

Тогда задача ЛП имеет вид:

$$\max\ 40x_1 + 30x_2$$

при ограничениях

$$\begin{cases} 3x_1 + 2x_2 \le 18, \\ 2x_1 + x_2 \le 10, \\ x_1 \ge 0,\ x_2 \ge 0. \end{cases}$$

Смысл ограничений:

- $3x_1 + 2x_2 \le 18$ — нельзя израсходовать больше древесины, чем есть;
- $2x_1 + x_2 \le 10$ — нельзя превысить имеющийся запас рабочего времени;
- $x_1, x_2 \ge 0$ — отрицательное число изделий производить нельзя.

5.11. Приближённое решение системы $Ax \approx b$

Иногда систему

$$Ax = b$$

нельзя решить точно: например, она переопределена или содержит ошибки измерений. Тогда ищут x, для которого невязка (ошибка)

$$Ax - b$$

как можно меньше.

Рассмотрим две постановки:

$$\min_x \|Ax - b\|_2 \qquad \text{и} \qquad \min_x \|Ax - b\|_1.$$

Норма ℓ_2: метод наименьших квадратов

Задача

$$\min_x \|Ax - b\|_2$$

имеет тот же набор решений, что и задача

$$\min_x \|Ax - b\|_2^2,$$

потому что квадрат — монотонная функция на $[0, \infty)$.

Именно для **квадрата нормы** выводятся **нормальные уравнения**:

$$A^T Ax = A^T b.$$

Почему получаются нормальные уравнения?

Рассмотрим функцию

$$f(x) = \|Ax - b\|_2^2 = (Ax - b)^T (Ax - b).$$

Необходимое условие минимума: все частные производные по переменным $x_1, \ldots, x_n$ равны нулю.

Вычисление этих производных приводит к равенству

$$2A^T(Ax - b) = 0.$$

Следовательно,

$$A^T(Ax - b) = 0,$$

то есть

$$A^T Ax = A^T b.$$

Это и есть нормальные уравнения.

Замечание. Если столбцы матрицы A линейно независимы, то матрица $A^T A$ невырождена, и решение единственно.

Оценка сложности для плотной матрицы $A \in \mathbb{R}^{m \times n}$:

- вычисление $A^T A$: $O(mn^2)$;
- вычисление $A^T b$: $O(mn)$;
- решение системы размера $n \times n$: $O(n^3)$;
- память: $O(n^2)$ для хранения $A^T A$.

Норма ℓ_1: сведение к линейному программированию

Задача

$$\min_x \|Ax - b\|_1$$

может быть сведена к задаче линейного программирования.

Напомним, что

$$\|Ax - b\|_1 = \sum_{i=1}^{m} |(Ax - b)_i|.$$

Чтобы убрать модуль, вводят новые переменные

$$t_1, \ldots, t_m \geq 0,$$

где каждая t_i должна быть не меньше абсолютной величины соответствующей невязки:

$$|(Ax - b)_i| \leq t_i.$$

Это эквивалентно двум линейным ограничениям:

$$-(t_i) \leq (Ax - b)_i \leq t_i.$$

Значит, задача переписывается так:

$$\min_{x,t} \sum_{i=1}^{m} t_i$$

при условиях

$$Ax - b \leq t, \qquad -(Ax - b) \leq t, \qquad t \geq 0.$$

Эквивалентно:

$$\min_{x,t} \mathbf{1}^T t$$

при

$$\begin{cases} Ax - t \leq b, \\ -Ax - t \leq -b, \\ t \geq 0. \end{cases}$$

Почему это работает На оптимуме каждая переменная t_i станет минимально возможной, то есть

$$t_i = |(Ax - b)_i|.$$

Поэтому целевая функция

$$\sum_{i=1}^{m} t_i$$

ровно совпадает с $\|Ax - b\|_1$.

Размер новой задачи

- новых переменных: m;
- новых линейных ограничений: $2m$;
- стоимость построения для плотного случая: $O(mn)$.

Пример 4: для ℓ_1

Пусть
$$A = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 4 \end{pmatrix}.$$
Тогда
$$Ax - b = \begin{pmatrix} x - 1 \\ 2x - 4 \end{pmatrix},$$
и задача
$$\min_x \|Ax - b\|_1$$
имеет вид
$$\min_x \big(|x - 1| + |2x - 4|\big).$$
Введём $t_1, t_2 \geq 0$. Тогда получаем ЛП:
$$\min_{x, t_1, t_2} t_1 + t_2$$
при условиях
$$\begin{cases} x - 1 \leq t_1, \\ -(x - 1) \leq t_1, \\ 2x - 4 \leq t_2, \\ -(2x - 4) \leq t_2, \\ t_1 \geq 0, \ t_2 \geq 0. \end{cases}$$
Это уже обычная задача линейного программирования без модулей.

Предложение 2

Как решить $\min c^T x$ при $Ax = b$, если переменные свободны по знаку

Идея решения

Если уже есть алгоритм, который умеет решать задачи вида
$$\min \ \tilde{c}^T z \qquad \text{при} \qquad \tilde{A} z = \tilde{b}, \qquad z \geq 0,$$
то нужно только избавиться от свободных переменных.

Для каждого $j = 1, \ldots, n$ вводим
$$x_j = x_j^+ - x_j^-, \qquad x_j^+ \geq 0, \quad x_j^- \geq 0.$$
Тогда
$$x = x^+ - x^-,$$
и исходная задача эквивалентна
$$\min \ c^T (x^+ - x^-)$$

при

$$A(x^+ - x^-) = b, \qquad x^+ \geq 0, \quad x^- \geq 0.$$

В матричном виде:

$$\min \begin{pmatrix} c \\ -c \end{pmatrix}^T \begin{pmatrix} x^+ \\ x^- \end{pmatrix}$$

при

$$\begin{pmatrix} A & -A \end{pmatrix} \begin{pmatrix} x^+ \\ x^- \end{pmatrix} = b, \qquad \begin{pmatrix} x^+ \\ x^- \end{pmatrix} \geq 0.$$

Это уже задача в канонической форме.

Размер новой задачи

- было n переменных, стало $2n$;
- число равенств осталось m.

Построение новой задачи требует $O(mn)$ операций для плотной матрицы.

6. TODO

7. TODO

8. TODO

9. TODO

10. TODO

11. TODO

12. TODO

13. TODO

14. TODO

15. TODO